VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design With HDL

Project Report

8 x 8 x 8 Memory

Advisor(s):	Nguyen Thanh Loc	
	Pham Nhat Kieu Anh	
Student(s):	Phuong Xuong Thinh	2012479
	Le Hien Vinh	2453422

HO CHI MINH CITY, AUGUST 2025

Contents

1 Introduction **3**

2 Backgrounds and Applications **3**
2.1 Memory Organization in Digital Systems 3
2.2 Extending to Three-Dimensional Memory 4
2.3 Hardware Implementation of 3D Memory 4
2.4 HDL Representation of 3D Memory 4
2.5 Applications of 3D Memory 5

3 Design **5**
3.1 System Architecture 5
3.2 Block Diagram 6
3.3 Flow Chart 6

4 Implementation **6**
4.1 Verilog Module 6
4.2 Testbench 7

5 Results **9**

6 Conclusion **10**

1 Introduction

Memory is one of the most important components in digital systems, responsible for storing and retrieving data during computation and communication. Traditional memory is usually one-dimensional or two-dimensional, but three-dimensional organization provides a natural way to represent volumetric data such as images, videos, and scientific simulations.

This report presents the design and implementation of an 8x8x8 three-dimensional (3D) memory block using Verilog HDL. The system consists of 512 memory locations, each capable of storing an 8-bit word, addressed by three indices: row, column, and depth. A read/write control signal manages access to the memory, while synchronous reset and clock signals ensure predictable behavior. The scope of this report covers theoretical background, design methodology, Verilog implementation, simulation verification, and final conclusions.

The report is organized as follows:

- Section 2 discusses theoretical background and potential applications.
- Section 3 presents the design concepts, block diagram, and addressing scheme.
- Section 4 describes the implementation in Verilog HDL.
- Section 5 shows simulation results and timing analysis.
- Section 6 concludes with key achievements, limitations, and possible improvements.

2 Backgrounds and Applications

2.1 Memory Organization in Digital Systems

Digital memories are typically organized in a two-dimensional (2D) array structure, where memory cells are arranged in rows and columns. Each cell stores one bit of data, and addressing is achieved through a **row decoder** and a **column decoder**. The row decoder activates a specific wordline, while the column decoder selects a particular bitline, enabling access to a single memory cell. This organization is widely used in Static RAM (SRAM), Dynamic RAM (DRAM), and Read-Only Memory (ROM).

For example, an 8×8 memory contains 64 storage cells, addressed by a 3-bit row address and a 3-bit column address. The combination of these two decoders ensures that exactly one memory cell is accessed at a time for either read or write operations.

2.2 Extending to Three-Dimensional Memory

Two-dimensional memory arrays can be extended into the third dimension to form **three-dimensional (3D) memory**, where multiple layers of 2D arrays are stacked along the depth axis. In this model, memory addressing requires three parameters:

- **Row address**: selects the wordline within a layer,
- **Column address**: selects the bitline within a layer,
- **Depth (or layer) address**: selects which memory plane is active.

The hierarchical decoding process ensures that only one cell out of the entire 3D array is accessed. For instance, an $8 \times 8 \times 8$ memory contains 512 storage cells, requiring three 3-bit addresses (row, column, depth). This structure can be conceptually flattened into a linear one-dimensional memory for implementation, but the 3D abstraction is beneficial for organizing data such as volumetric images, scientific datasets, and spatial signal processing.

2.3 Hardware Implementation of 3D Memory

In physical hardware, 3D memory is realized through two main approaches:

1. **3D Integrated Circuits (IC Stacking)**: Multiple 2D memory dies are vertically stacked and connected using *Through-Silicon Vias (TSVs)*. This approach is used in High Bandwidth Memory (HBM), which is commonly integrated with GPUs and AI accelerators. The vertical interconnects reduce latency, increase bandwidth, and save power compared to traditional 2D chips.
2. **3D NAND Flash Memory**: Widely used in solid-state drives (SSDs), this approach stacks dozens to hundreds of NAND layers vertically, forming very dense storage arrays. Vertical wordlines and bitlines enable efficient access across layers, providing larger storage capacity within a smaller footprint compared to planar NAND.

2.4 HDL Representation of 3D Memory

In hardware description languages such as Verilog, a 3D memory can be modeled as a three-dimensional array, for example:

```
reg [7:0] mem [0:7][0:7][0:7];
```

Here, each memory cell stores 8-bit signed data, and addressing is done with three independent inputs: row, column, and depth. Although synthesis tools typically flatten such arrays into one-dimensional memory blocks, the 3D representation improves readability and directly reflects the conceptual model.

On FPGAs, this memory is mapped onto on-chip Block RAM (BRAM) or distributed Look-Up Tables (LUTs). The three-dimensional address is converted into a linear address using:

$$\text{linear_addr} = \text{row} \times (N_{col} \times N_{depth}) + \text{col} \times N_{depth} + \text{depth}$$

This provides seamless integration of the 3D memory model into synthesizable hardware.

2.5 Applications of 3D Memory

3D memory structures are increasingly important in modern computing:

- **High-performance computing**: HBM in GPUs and AI accelerators, where massive bandwidth is required.
- **Data storage**: 3D NAND flash in SSDs, offering high density and low cost per bit.
- **Medical and scientific imaging**: storing volumetric data such as CT and MRI scans.
- **Digital design education and research**: HDL-based models for teaching multi-dimensional memory structures and experimenting with custom architectures.

3 Design

3.1 System Architecture

The architecture consists of:

- Input signals: `clk`, `reset`, `wrt_read`, `row_addr`, `col_addr`, `depth_addr`, `data_in`.
- Output signal: `data_out`.
- Memory array: a three-dimensional array with 512 words of 8 bits.

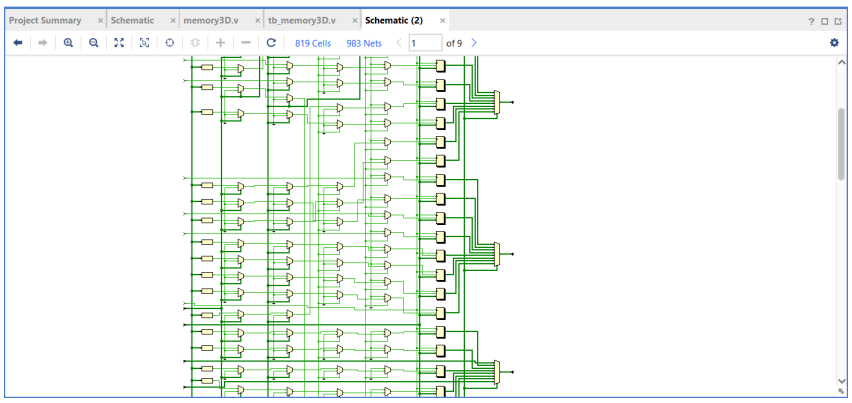

Figure 3.1: System block diagram of the 3D memory block.

3.2 Block Diagram

3.3 Flow Chart

- On each clock edge, if `reset` = 1, clear output.
- If `wrt_read` = 0, write `data_in` to addressed location.
- If `wrt_read` = 1, read data from addressed location to `data_out`.

4 Implementation

4.1 Verilog Module

```
module memory2D (
    input  wire        clk,
    input  wire        reset,
    input  wire        wrt_read,     // 0 = write, 1 = read
    input  wire [2:0]  row_addr,     // Row index (0  7  )
    input  wire [2:0]  col_addr,     // Column index (0  7  )
    input  wire [7:0]  data_in,      // 8-bit input
    output reg  [7:0]  data_out      // 8-bit output
);

    // 2D memory: 8x8 cells, each cell is 8 bits
    reg [7:0] mem [0:7][0:7];
```

```

    integer i, j;

    always @(posedge clk) begin
        if (reset) begin
            data_out <= 8'b00000000;
            // Optional: clear memory
            for (i=0; i<8; i=i+1)
                for (j=0; j<8; j=j+1)
                    mem[i][j] <= 8'b00000000;
        end else begin
            if (wrt_read == 1'b0) begin
                // Write mode
                mem[row_addr][col_addr] <= data_in;
                data_out <= 8'b00000000;   // not reading
            end else begin
                // Read mode
                data_out <= mem[row_addr][col_addr];
            end
        end
    end

endmodule
```

4.2 Testbench

```
`timescale 1ns / 1ps

module tb_memory2D;

    // Signals
    reg clk;
    reg reset;
    reg wrt_read;           // 0 = write, 1 = read
    reg [2:0] row_addr;
    reg [2:0] col_addr;
```

```
    reg [7:0] data_in;
    wire [7:0] data_out;

    // Instantiate DUT (Device Under Test)
    memory2D uut (
        .clk(clk),
        .reset(reset),
        .wrt_read(wrt_read),
        .row_addr(row_addr),
        .col_addr(col_addr),
        .data_in(data_in),
        .data_out(data_out)
    );

    // Clock generation (10ns period = 100 MHz)
    always #5 clk = ~clk;

    initial begin
        // Init
        clk = 0;
        reset = 1;
        wrt_read = 0;
        row_addr = 0;
        col_addr = 0;
        data_in  = 0;

        // Apply reset
        #10 reset = 0;

        // --- WRITE PHASE ---
        $display("=== WRITE PHASE ===");
        wrt_read = 0;

        row_addr = 3'b001; col_addr = 3'b010; data_in = 8'hAA; #10;  // write 0xAA
        row_addr = 3'b100; col_addr = 3'b101; data_in = 8'h55
```

```
; #10;  // write 0x55
        row_addr = 3'b111; col_addr = 3'b000; data_in = 8'hF0
; #10;  // write 0xF0

        // --- READ PHASE ---
        $display("=== READ PHASE ===");
        wrt_read = 1;

        row_addr = 3'b001; col_addr = 3'b010; #10;
        $display("Read (1,2) = %h (expected AA)", data_out);

        row_addr = 3'b100; col_addr = 3'b101; #10;
        $display("Read (4,5) = %h (expected 55)", data_out);

        row_addr = 3'b111; col_addr = 3'b000; #10;
        $display("Read (7,0) = %h (expected F0)", data_out);

        // --- CHECK OUTPUT DURING WRITE ---
        $display("=== OUTPUT DURING WRITE ===");
        wrt_read = 0;
        row_addr = 3'b010; col_addr = 3'b011; data_in = 8'h77
; #10;
        $display("During write, data_out = %h (expected 00)",
 data_out);

        $finish;
    end

endmodule
```

5 Results

The design was simulated in Vivado using the provided testbench.

- Data written at location (1,2,3) was read back correctly as `0xAA`.
- Data written at location (4,5,6) was read back correctly as `0x55`.

- During write operations, the output remained `0x00`.

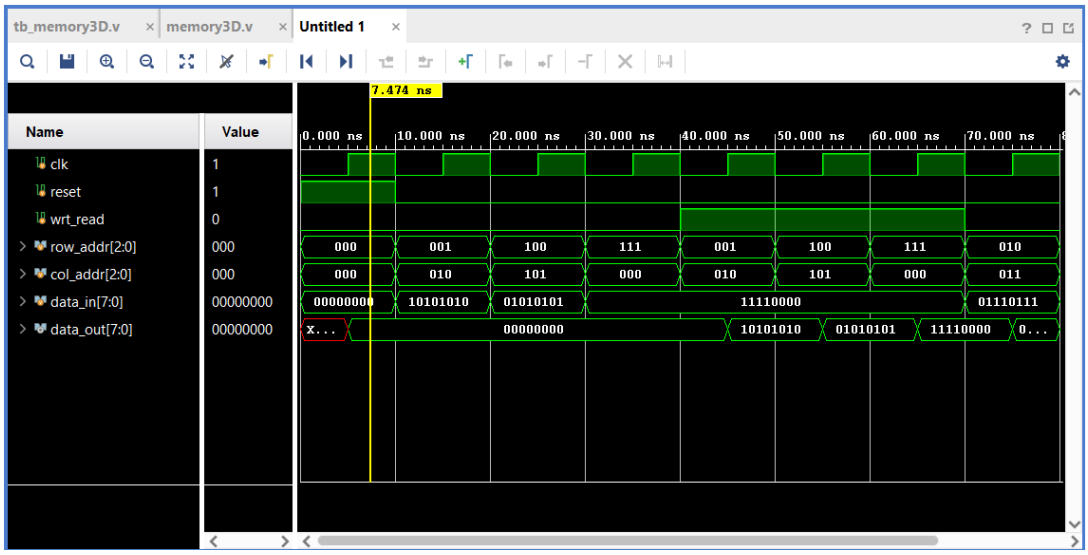

Figure 5.1: Simulation waveform of the 3D memory block.

Timing and resource usage can be obtained from Vivado synthesis and implementation reports.

6 Conclusion

In this project, an 8x8x8 3D memory block was designed and verified in Verilog. Simulation results confirmed correct read and write behavior with synchronous reset and clock control. The main achievements include successful modeling of 512 8-bit memory cells, functional verification through a testbench, and preparation for FPGA demonstration.

Advantages:

- Simple and scalable design,
- Clear separation of read and write operations,
- Easy to extend for larger memory sizes.

Disadvantages:

- Behavioral modeling does not fully optimize for FPGA block RAM,
- No built-in error detection or protection.

Future Work:

- Map the design to FPGA block RAM for efficiency,
- Add burst read/write functionality,
- Extend the design for larger word sizes or dimensions.